Phase 6: controlled source tasks

Phase 6 adds the **Source tasks** workspace. You can prepare a private copy of explicitly permitted source files, document behavioral contracts, review an exact patch, stage it privately, and roll it back while preserving the original checkout.

Current checkpoint: source-task implementation and the local Docker sandbox are available. A controlled Java/Maven fixture compiled and passed its test offline inside the restricted container. Customer-specific dependency caches, artifact reproduction and the broader Phase 6 security/characterization gates still require validation. Build buttons also require the workspace build policy to be enabled. This is not a completed modernization or a production-ready transformation engine. See verification.

1. Prepare evidence and access

1. Open ArchIntel at `http://127.0.0.1:8000` and sign in. Use `Start-ArchIntel.cmd` from the application folder if the local services are stopped.
2. Complete a static analysis. In **Evidence & digital twin**, capture an architecture version. In **Modernization intelligence**, save and review a planning proposal as `APPROVED_PLANNING`.
3. In **Security & access**, register an existing local Git checkout using the full 40-character commit. Its application must match the architecture version. The checkout must be below an operator-configured source root, with its own normal `.git` directory.
4. Give your identity `ANALYSIS_READ`, `ANALYSIS_WRITE` and `SOURCE_READ`, plus an explicit repository grant for the files involved. Runtime-backed versions also require runtime-read permission. Administrators need source grants too.
5. A security administrator enables source reads and **Source tasks** in workspace policy. **Isolated builds** is a separate permission. Defaults remain disabled; installing the feature does not grant access or change existing policy.

Use committed source matching the analyzed artifact. Task preparation reads committed blobs; uncommitted customer edits are preserved and are not copied into the task. Repository registration alone does not prove that its source produced the analyzed artifact.

2. Prepare the private workspace

In **Source tasks**, select the application, captured version, approved proposal and decision. Enter:

Field	What to enter
Repository ID and pinned commit	The registered source repository and its exact commit
Objective and expected change	A specific, reviewable intent
Snapshot files	Explicit repository-relative file paths, one per line
Files allowed to change	The narrower set permitted in a patch; new paths may be declared here
Built artifact path	A JAR or WAR under `target/`, for example `target/application.jar`

Click **Prepare isolated task**. Check the returned state and prerequisites. `PLANNED` means the authorized snapshot was prepared. `FAILED` with a preparation error requires storage/Git investigation and a new task after repair. A retry of the same creation request does not create duplicate tasks; the UI generates a new request key after a successful response.

The private task directory has its own Git repository and baseline copy. It does not share Git worktree metadata with the customer's checkout. A partial snapshot can be reviewed, but it cannot run a baseline build: the selected file set must equal the full committed repository file set.

Supported inputs are regular UTF-8 text files with portable ASCII paths. Binary assets, links/junctions, submodules, `.gitattributes`, `.gitmodules`, `.lfsconfig`, dotenv files and common private-key files are unsupported or blocked. A repository containing such files cannot currently meet the complete-snapshot build gate. Default limits are 100 files / 1 MiB; the policy API can raise them up to 2,000 files / 32 MiB. Filters can reject legitimate source containing sensitive-looking sample data; do not disable safeguards to force it through.

3. Describe and review behavior

Open **Contracts**. Enter a behavior name, captured test-file path, exact JUnit case name (`ClassName#methodName`), scenario and expected result. Add the contract, then enter a review note and select **Review contract**.

A reviewed description is a human assertion. It becomes baseline test evidence only when a passing baseline build reports that exact non-skipped test case. Critical contracts without such evidence remain explicit gaps. Existing tests can still miss important business behavior; the application does not automatically infer or generate a complete test suite.

4. Propose and inspect a patch

1. Enter an operation/review note of at least eight characters.
2. Open **Patch review**, choose an allowed file and operation (`MODIFY`, `ADD` or `DELETE`).
3. For an existing file, select **Load current file**, then enter the complete replacement content. A deletion has no replacement content.
4. Select **Propose changeset**, then **Inspect patch**. Review the exact diff, file list, risk and patch hash.
5. Select **Approve isolated staging** or **Reject patch**. A high-risk patch needs a different human security administrator with the necessary source grant. Build/dependency/configuration/test/security-sensitive files, deletions and large changes trigger the conservative high-risk rule.
6. Select **Stage reviewed patch**. The task becomes `STAGED`; only its private files change.

Approval is bound to the patch, source hashes, reviewer authorization and policy revision, and expires after 24 hours. A changed policy, revoked grant, stale task revision or changed files requires refreshed review. Each task supports one initial patch against its baseline; prepare a new task for another staged revision.

Staging approval permits a private patch inspection step. It does not certify behavior, authorize merge, or approve deployment. Source-task changes are currently supplied by a person or API client; this phase does not generate AI refactors.

5. Run builds when the environment is available

An operator must provision a local Docker engine in Linux-container mode and a trusted immutable image containing the required JDK, Maven and a complete offline dependency/plugin cache. See the developer guide. The runner does not pull images or install dependencies automatically.

The **Builds** tab offers baseline and candidate runs. Both use a fixed offline Maven package recipe inside the container. The baseline must run non-skipped tests successfully and reproduce the exact SHA-256 of the artifact already analyzed. A different timestamp or other non-reproducible build input may cause an artifact mismatch; do not override the hash gate. Establish a reproducible build first.

`READY` means the baseline and reviewed critical-contract prerequisites passed. A candidate build requires a staged patch. Even a passing candidate build leaves the task `VALIDATION_BLOCKED`: architecture reanalysis, comprehensive generated-code security checks and final transformation approval are not delivered by this slice.

Structured build results show counts, artifact hash and error codes. Raw customer build logs are not retained. Only single-module Maven projects with Surefire reports in `target/surefire-reports` are supported by this initial adapter. External-service integration tests, multi-module report aggregation and Gradle are not supported.

6. Pause, recover and roll back

- **PAUSE** prevents further task progression. It does not immediately terminate an already running container command; that command remains time-bounded. Wait for its recorded completion before resuming or rolling back.
- **RESUME** rechecks current evidence and workspace integrity. It does not bypass prerequisites.
- **RECHECK** reevaluates baseline/contract prerequisites in supported states.
- **ROLLBACK** restores the private task files to their captured baseline and ends that task. The customer's original files, branch and commit remain untouched.

Unexpected files, changed baseline data or unreviewed byte changes cause integrity checks to refuse overwrite. Interrupted preparation, staging or builds are marked `FAILED` at API startup; they are not automatically retried. An interrupted approved staging operation can be rolled back when all surviving bytes match the recorded before/after hashes. Other damage requires operator inspection.

7. Troubleshooting and verification

Symptom	Meaning / next step
Isolated builds unavailable	Install/configure the local environment and immutable image; no host fallback exists
403 / source task disabled	Check source-read permission, task policy, explicit grant and protected paths
409 / task changed	Refresh, inspect the current revision, then review again
Incomplete source snapshot	Include every supported committed file within the authorized policy budget
Baseline artifact mismatch	Source/build inputs do not reproduce the analyzed artifact
Critical behavior unvalidated	Review the contract and run a matching successful baseline test
Workspace integrity failure	Inspect unexpected files or interrupted writes; do not overwrite unrelated work
Validation blocked after candidate build	Required architecture/security validation remains pending

For a demonstration, verify preparation, exact patch review, private staging and rollback on a controlled repository. Confirm the original checkout remains unchanged. Do not present a successful patch workflow as evidence of a completed real build, validated modernization or production deployment.

Developer handoff | Verification record